

Aula 4 — PWA Offline-First

"Instale uma vez, funcione em qualquer lugar — mesmo sem internet"

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 01/07/2026

Recap da Aula 3

- Flutter: widgets, Stateless vs Stateful, Navigator
- Compose Multiplatform — compartilhando UI entre Android e iOS
- Estudos de caso Airbnb, Discord, Coinbase

Atividade 3 — entregue?

Agenda da aula (3h30)

Bloco 1 (75min) — PWA: o que é, por que importa, Web App Manifest, SW lifecycle

→ Conceitos · Manifest · Service Worker · Cache Storage

Intervalo (30min)

Bloco 2 (75min) — PWA offline-first na prática

→ Workbox · IndexedDB · Lighthouse · **Atividade 4 scaffold** · Demo Flutter+PWA

| KMP movido para **Aula 5 (08/07)** junto com BFF/GraphQL.

Fechamento (30min) — BFF/GraphQL: conectando os dois mundos

→ Demo CineHub · enunciado **A4**

Objetivos de aprendizagem

KMP:

- Compartilhar lógica de negócio entre Android, iOS e Web com Kotlin
- Integrar HTTP com Ktor + kotlinx.serialization + TMDB API
- Entender `expect` / `actual` para código platform-specific

PWA:

- Compreender ciclo de vida de Service Workers (install, activate, fetch)
- Aplicar cache strategies com Workbox (NetworkFirst, CacheFirst)
- Converter SPA em app instalável e offline-first

BFF:

- Entender padrão Backend-for-Frontend — um servidor, múltiplos clientes
- Comparar REST (KMP/PWA) vs GraphQL (capstone CineHub)

KotlinProject — como foi criado

Projeto gerado pelo **KMP wizard oficial** da JetBrains:

kmp.jetbrains.com

Config usada: Android ✓ · iOS ✓ · iOS UI = **Compose** · Include Tests ✓

Essa URL reproduz exatamente o starter. Qualquer aluno pode gerar o mesmo projeto do zero no wizard — é o ponto de partida oficial pra projetos KMP novos.

O que vamos rodar hoje — KotlinProject

Três apps, um módulo compartilhado (`shared` — Compose Multiplatform):

App	Módulo	Plataforma
<code>androidApp</code>	<code>:androidApp</code>	Android
<code>iosApp</code>	<code>iosApp/</code>	iOS (Xcode project)
<code>webApp</code>	<code>:webApp</code>	Web (JS / Wasm)

Targets declarados em `shared/build.gradle.kts`:

`androidTarget`, `iosArm64`, `iosSimulatorArm64`, `js`, `wasmJs`

Toolchain verificado — versões exatas

Ferramenta	Versão
Android Studio	Quail 2026.1.1 Patch 2
Gradle wrapper	9.1.0
Kotlin	2.4.0
AGP	9.0.1
Compose Multiplatform	1.11.1
JDK (terminal)	OpenJDK 17.0.15
JDK (Studio)	bundled OpenJDK 21.0.10
Xcode	26.1.1

Projeto **não carrega** no Studio < Quail 2026.1.1. Atualizar se necessário.

Setup — pré-requisitos

```
# 1. Verificar Xcode CLI  
xcode-select -p  
# → /Applications/Xcode.app/Contents/Developer ✓
```

- **macOS** + Homebrew (/opt/homebrew/bin/brew)
- **Android Studio Quail 2026.1.1+** — versão exata importa
- **Xcode 26.x** com command-line tools selecionados
- `local.properties` criado automaticamente pelo Studio:

```
sdk.dir=/Users/<seu-user>/Library/Android/sdk
```

| Não commitar — é caminho de máquina.

Setup — plugin KMP no Android Studio

A parte mais traiçoeira. A configuração `iosApp` só aparece depois que o plugin carrega.

Passo a passo:

1. **Settings → Plugins → Marketplace** → buscar **Kotlin Multiplatform** (JetBrains s.r.o.) → Install
2. Se aparecer erro **Requires plugin 'com.intellij.marketplace' to be installed** + badge vermelho:
 - Marketplace → buscar **JetBrains Marketplace Licensing** → Install
3. Reiniciar o Studio
4. Dropdown de run config agora mostra `iosApp` ao lado de `androidApp` ✓

`kdoctor` pode reportar "KMM plugin not installed" — é falso positivo (procura o id legado 14936). Ignorar.

Setup — iOS Simulator (se não existir)

```
# Checar devices disponíveis
xcrun simctl list devices available

# Se a lista estiver vazia sob um runtime, criar:
xcrun simctl create "iPhone 16" \
    com.apple.CoreSimulator.SimDeviceType.iPhone-16 \
    com.apple.CoreSimulator.SimRuntime.iOS-26-1
```

Sintoma sem device: kdoctor avisa *"Couldn't find available Apple Simulators"*.

Verificar ambiente — kdoctor

```
brew install kdoctor  
kdoctor           # ou /opt/homebrew/bin/kdoctor
```

Esperado: OS ✓ · Java ✓ · Xcode ✓

2 warnings seguros para ignorar:

- *"Android Studio: KMM plugin not installed"* — kdoctor 1.1.0 checa id legado; nosso plugin está ok
- *"CocoaPods not configured"* — este projeto não tem `Podfile` ; só necessário com libs CocoaPods

Rodando os 3 targets

Android

- Dropdown → `androidApp` + `emulador/device` → 

iOS

- Dropdown → `iosApp` + `iPhone 16` → 
- Fallback (sem plugin):

```
open iosApp/iosApp.xcodeproj    # Run pelo Xcode
```

Web

```
./gradlew :webApp:jsBrowserDevelopmentRun    # JS  
./gradlew :webApp:wasmJsBrowserDevelopmentRun    # Wasm
```

Gotchas do KotlinProject

- **Apple Silicon:** target é `iosSimulatorArm64`. `iosX64` **não** é declarado → simuladores Intel não buildam.
- **Config cache com comportamento estranho:**

```
./gradlew --no-configuration-cache <task>
```

- **Nunca commitar** `local.properties` — caminho do SDK é específico de máquina.

Kotlin Multiplatform (KMP) — Stack & Forças

Stack:

- **Kotlin** linguagem em Android, iOS via Kotlin/Native, JS, JVM, Linux, Windows
- **expect / actual** — contratos compartilhados, implementações por plataforma
- **Compose Multiplatform** — UI compartilhada (em maturação)
- **Ktor** — HTTP client
- **kotlinx.serialization** — JSON

Forças:

- Compartilha **regra de negócio**, não UI (na maioria dos casos)
- Excelente integração com Android nativo
- iOS via Swift interop

Kotlin Multiplatform (KMP) — Fraquezas

- Talent pool ainda restrito
- iOS interop com debugging chato
- Compose Multiplatform ainda jovem em iOS

Kotlin — sintaxe em 1 slide

```
val nome = "Ana"           // imutável (≈ const)
var contador = 0           // mutável
fun saudar(n: String) = println("Oi, $n") // função

data class Filme(val titulo: String, val nota: Double) // equals/copy/toString grátis

val apelido: String? = null // ? = pode ser null
val tam = apelido?.length ?: 0 // safe-call (?.) + elvis (?:)
listOf(1, 2, 3).filter { it > 1 } // lambda: it = o item
```

`val / var` · `data class` = seu modelo · `? / ?:` = null-safety · `{ it }` = lambda. É a linguagem do "shared" do KMP.

KMP — arquitetura (shared + targets)

commonMain — regra de negócio compartilhada

models, repositórios, casos de uso · `expect` declara o que muda por plataforma

▼ compila pra cada target ▼

androidMain

`actual` → OkHttp, APIs Android

iosMain

`actual` → URLSession, APIs iOS

Compartilha **lógica**, não UI. O que é específico de plataforma fica nos `actual`.

KMP — `expect` / `actual`

```
// commonMain
expect class HttpClient {
    suspend fun get(url: String): String
}

// androidMain
actual class HttpClient {
    actual suspend fun get(url: String): String {
        // OkHttp implementation
    }
}

// iosMain
actual class HttpClient {
    actual suspend fun get(url: String): String {
        // NSURLSession implementation
    }
}
```

`expect` declara o contrato. `actual` implementa por plataforma.

Hands-on guiado — Kotlin (Playground)

play.kotlinlang.org (zero instalação). Escreva a "shared logic" dos favoritos — a mesma do `favoritesStore` do app:

```
data class Movie(val id: Int, val title: String, val rating: Double) // 1. modelo

fun favorites(movies: List<Movie>, favIds: Set<Int>) = // 2. função pura
    movies.filter { it.id in favIds }

fun main() {
    val all = listOf(
        Movie(1, "Matrix", 8.7),
        Movie(2, "Inception", 8.8),
        Movie(3, "Tenet", 7.4),
    )
    val favs = favorites(all, setOf(1, 3)) // 3. rode!
    println(favs.map { it.title }) // → [Matrix, Tenet]
}
```

(1) `data class` = modelo · (2) função pura de filtro · (3) rode. Essa lógica o KMP compartilha entre Android e iOS — só o que toca plataforma (rede, sensores) vira `expect / actual`.

Estudos de caso — KMP

KMP:

- Square (Cash App)
- JetBrains (próprio Toolbox, Fleet)
- McDonald's mobile
- Philips

Intervalo — 30min

Volta às :____

Aproveita pra:

- Esticar perna
- Confirmar emulador Android rodando (projeto KMP no AS)
- Pegar token TMDB se não tiver: developer.themoviedb.org → API → Bearer

Próximo bloco: PWA com Service Workers. Chrome DevTools aberto na aba **Application**.

"PWA é só web?" — fato vs ficção

Twitter Lite (*Eng Blog, 2017*): offline real, 75% mais tweets enviados.

Pinterest (*web.dev, 2017*): 60% mais engajamento, instalável.

Spotify Web (2023): PWA instalável no desktop — mesmo conteúdo do app.

Forbes (2016): 100ms de melhoria no TTI → +1% de sessões.

PWA não é cosmético. Mas não substitui nativo em todos os casos — limitações reais no iOS existem (sem push até iOS 16.4, sem Background Sync). Próximos slides detalham.

⚠️ Dados acima: blogs de engenharia das próprias empresas (não estudos controlados independentes).

Anatomia de uma PWA

Uma PWA precisa de **3 componentes**:

1. **Web App Manifest** (`manifest.json`) — metadados (nome, ícones, cores, display)
2. **Service Worker** — interceptador de network, cache, offline
3. **HTTPS** — origem segura obrigatória

Bonus para "instalável":

- Ícones em múltiplos tamanhos
- `start_url`, `display: standalone`
- Service Worker com `fetch handler`

Web App Manifest

```
{
  "name": "Minha PWA",
  "short_name": "MinhaApp",
  "description": "Descrição curta da app",
  "start_url": "/",
  "scope": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#003366",
  "icons": [
    {
      "src": "/icons/icon-192.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/icons/icon-512.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "any maskable"
    }
  ]
}
```

Por que o PWA funciona offline?

"Para exibir qualquer coisa, o navegador precisa de HTML, CSS e JS. A mágica do PWA não é fazer esses arquivos aparecerem do nada — é como ele os armazena no seu dispositivo."

O segredo tem nome: Service Worker

- Script JS que roda **em segundo plano**, separado da aba
- Age como um **proxy de rede local**: intercepta todas as requisições
- Decide: *"pego da internet ou sirvo do cache?"*

Como o PWA funciona offline — passo a passo

Etapa	O que acontece
1. 1º acesso (com internet)	Browser baixa HTML, CSS, JS, imagens + instala o Service Worker
2. Cache Storage	SW copia os assets críticos pra uma "despensa" local no navegador
3. Offline (avião ✈️)	SW intercepta as requisições → serve os arquivos da "despensa"

O app **abre instantaneamente** porque o navegador lê o HTML e o JS do cache local — sem tocar a rede.

E os dados dinâmicos? — IndexedDB

O Cache Storage traz o **esqueleto** (HTML/CSS/JS).

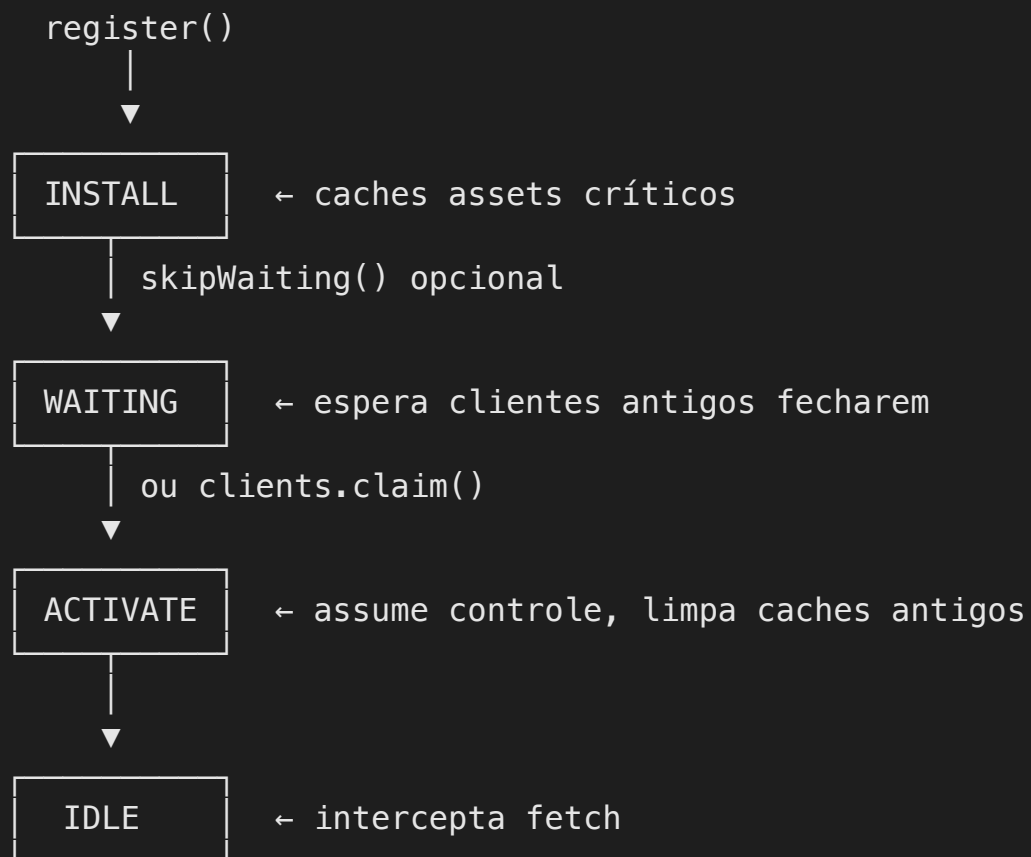
O IndexedDB traz o **conteúdo** (seus dados pessoais).

Camada	Armazena	Quando preenche
Cache Storage (SW)	HTML, CSS, JS, imagens	Instalação / 1ª visita
IndexedDB	Filmes, favoritos, posts	Ao usar o app online

No A4: `moviesRepository.ts` salva a resposta da API no IndexedDB.

Offline → SW serve o app → React lê os filmes do IndexedDB. ✓

Service Worker — lifecycle



Service Worker — install + activate

```
// sw.js
const CACHE_NAME = 'app-v3';
const PRECACHE_URLS = ['/', '/index.html', '/styles.css', '/app.js'];

self.addEventListener('install', (event) => {
  event.waitUntil(
    caches.open(CACHE_NAME).then((cache) => cache.addAll(PRECACHE_URLS))
  );
  self.skipWaiting(); // assume controle imediato (cuidado!)
});

self.addEventListener('activate', (event) => {
  event.waitUntil(
    caches.keys().then((names) =>
      Promise.all(
        names.filter((n) => n !== CACHE_NAME).map((n) => caches.delete(n))
      )
    )
  );
  self.clients.claim();
});
```


Cache strategies

Cache-first (estáticos)

Tenta cache; se falhar, vai pra rede; cacheia o resultado.

Network-first (dados dinâmicos)

Tenta rede com timeout; se falhar, vai pro cache.

Stale-While-Revalidate

Serve cache imediatamente (rápido) + busca rede em paralelo (atualiza próxima request).

Cache-only / Network-only

Casos específicos (assets imutáveis vs telemetria).

Cache-first — implementação

```
self.addEventListener('fetch', (event) => {  
  if (event.request.url.includes('/static/')) {  
    event.respondWith(  
      caches.match(event.request).then((cached) => {  
        return cached || fetch(event.request).then((response) => {  
          const clone = response.clone();  
          caches.open(CACHE_NAME).then((cache) => cache.put(event.request, clone));  
          return response;  
        });  
      });  
    );  
  }  
});
```

Use para JS, CSS, imagens, fonts versionadas.

Stale-While-Revalidate — implementação

```
self.addEventListener('fetch', (event) => {  
  if (event.request.url.includes('/api/feed')) {  
    event.respondWith(  
      caches.open(CACHE_NAME).then(async (cache) => {  
        const cached = await cache.match(event.request);  
        const fetchPromise = fetch(event.request).then((response) => {  
          cache.put(event.request, response.clone());  
          return response;  
        });  
        return cached || fetchPromise;  
      })  
    );  
  }  
});
```

Usuário vê algo imediatamente, próxima carga já está atualizada.

O que é o Workbox?

Workbox é uma biblioteca do Google que abstrai a API de Service Worker.

Sem Workbox, você escreve cache manual para cada rota:

```
self.addEventListener('fetch', event => {  
  event.respondWith(  
    caches.match(event.request).then(cached => cached || fetch(event.request))  
  );  
});
```

Com Workbox, você declara a estratégia:

```
registerRoute(/\.png$/, new CacheFirst());
```

Sem Workbox	Com Workbox
~150 linhas de boilerplate	~10 linhas declarativas

Workbox é para Service Workers o que o Axios é para `fetch` — você podia fazer sem, mas não vale a pena.

Workbox — abstração madura

```
import { precacheAndRoute } from 'workbox-precaching';
import { registerRoute } from 'workbox-routing';
import {
  CacheFirst,
  NetworkFirst,
  StaleWhileRevalidate,
} from 'workbox-strategies';

precacheAndRoute(self.__WB_MANIFEST);

registerRoute(/\.(?:png|jpg|svg)$/ , new CacheFirst({ cacheName: 'images' }));
registerRoute(
  /\api\/static\/ ,
  new StaleWhileRevalidate({ cacheName: 'api-static' })
);
registerRoute(/\api\/dynamic\/ , new NetworkFirst({ cacheName: 'api-dynamic' }));
```

Workbox elimina boilerplate. Use em produção.

IndexedDB — persistência local estruturada

`localStorage` é síncrono e limitado (~5MB). **IndexedDB** é a opção real.

```
import Dexie from 'dexie';

const db = new Dexie('MyAppDB');
db.version(1).stores({
  drafts: '++id, content, createdAt',
  user: '&id, name, email',
});

// Add
await db.drafts.add({ content: 'Hello', createdAt: Date.now() });

// Query
const recent = await db.drafts
  .orderBy('createdAt')
  .reverse()
  .limit(10)
  .toArray();
```

Dexie torna IndexedDB usável.

Background Sync API

Usuário fecha app sem conexão. Operação fica pendente. Quando conectividade volta, **Service Worker dispara sync event mesmo com app fechado.**

```
// Page side
async function submitOfflineSafe(message) {
  await db.outbox.add({ message, attempts: 0 });

  if ('serviceWorker' in navigator && 'SyncManager' in window) {
    const registration = await navigator.serviceWorker.ready;
    await registration.sync.register('flush-outbox');
  }
}

// sw.js
self.addEventListener('sync', (event) => {
  if (event.tag === 'flush-outbox') {
    event.waitUntil(flushOutbox());
  }
});
```

Web Worker vs Service Worker — quando usar cada um?

Web Worker — "preciso processar dados sem travar a UI"

```
// Parsing de CSV gigante sem congelar a página  
const worker = new Worker('parser.js');  
worker.postMessage(csvData);           // envia pra thread separada  
worker.onmessage = e => render(e.data); // recebe resultado
```

🎯 Casos: filtrar 50k registros, aplicar filtro em imagem, rodar modelo ML

Service Worker — "preciso interceptar rede e controlar cache"

```
// Responde a fetch do app inteiro, mesmo sem internet
self.addEventListener('fetch', event => {
  event.respondWith(
    caches.match(event.request) || fetch(event.request)
  );
});
```

🎯 Casos: carregar app offline, cachear TMDB, push notification, sync em background

Web Worker = **thread paralela** pra sua página.

Service Worker = **proxy de rede** que persiste entre sessões.

Web Workers vs Service Workers

Aspecto	Web Worker	Service Worker
Propósito	Compute pesado off-thread	Network proxy + cache
Lifecycle	Scope da página	Persistente, instalado
Escopo	Página atual	Origin/scope inteiro
API	postMessage com page	fetch , cache , push , sync
Múltiplas instâncias	1 por chamada	1 por scope

Use Web Worker para parsing JSON gigante, image processing, ML em browser.

Lighthouse CI

```
# .github/workflows/lighthouse.yml
name: Lighthouse CI
on: [pull_request]

jobs:
  lighthouse:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22 }
      - run: npm ci
      - run: npm run build
      - run: npm install -g @lhci/cli@0.13.x
      - run: lhci autorun
```

```
// lighthouseci.json
{
  "ci": {
    "assert": {
      "assertions": {
        "categories:performance": ["error", { "minScore": 0.9 }],
        "categories:pwa": ["error", { "minScore": 0.9 }]
      }
    }
  }
}
```

PWA vs RN-Web vs Capacitor

Aspecto	PWA	RN-Web	Capacitor
Distribuição	URL ou install	URL ou install	App Store + Play
Capacidade nativa	Limitada	Limitada	Total via plugins
Tamanho bundle	Pequeno	Médio	Maior
Performance	Web	Web (com RN-style)	Nativo wrapped
Update	Instantâneo	Instantâneo	Via store

Cada um cobre nichos. Não competem diretamente.

BFF — Backend-for-Frontend

O problema sem BFF

- RN, KMP, Web cada um bate em endpoints REST diferentes
- Over-fetching (campos que não usa) e under-fetching (N+1 requests)
- Múltiplos times sincronizando formatos de resposta

Com BFF + GraphQL

- Um endpoint `/graphql` serve **todos os clientes**
- Cliente pede **só os campos que precisa**
- Schema = contrato único versionado

```
# Android pergunta isso:
query {
  popularMovies {
    results { title voteAverage }
  }
}

# PWA pergunta isso:
query {
  popularMovies {
    results { title posterPath overview }
  }
}
```

Mesma query shape, respostas diferentes. O BFF otimiza por cliente sem mudar o backend.

GraphQL na prática — o servidor CineHub

```
# No terminal — pasta compartilhado/capstone/server  
npm install && npm run seed && npm start  
# → CineHub GraphQL pronto em http://localhost:4000/
```

Abra <http://localhost:4000/> no browser — Playground Apollo Studio.

Queries disponíveis:

```
{ popularMovies(page: 1) { page results { id title voteAverage } } }  
  
{ searchMovies(query: "Matrix") { id title releaseDate } }  
  
{ movie(id: 550) { title overview } }
```

O CineHub tem dados de filmes clássicos no SQLite local. Sem TMDB token necessário — ótimo pra demo.

TMDB REST vs GraphQL — dois sabores do mesmo problema

	TMDB REST (exercícios)	CineHub GraphQL (demo)
Autenticação	Bearer token no header	JWT local (register/login)
Popular movies	GET /movie/popular	query { popularMovies }
Busca	GET /search/movie?query=x	query { searchMovies(query:"x") }
Over-fetching	Sim (resposta fixa)	Não (cliente escolhe campos)
Offline	Cache HTTP / SW	Normalização client-side

No KMP usamos Ktor + TMDB REST — simples, sem servidor local.

Na PWA usamos fetch + TMDB REST + Service Worker para offline.

No BFF real (capstone) tudo passa pelo GraphQL.

Atividade 6 — KMP: filmes populares (15 pts)

Stack: KotlinProject (Android) + Ktor + TMDB REST API

O que você vai implementar no `shared/src/commonMain/` :

```
data/  
  Movie.kt      ← @Serializable (já pronto)  
  TmdbApi.kt    ← TODO 1: implementar popularMovies() com Ktor  
  App.kt        ← TODO 2: LaunchedEffect + TODO 3: LazyColumn
```

Setup:

```
cd exercicios/06-kmp/pratica  
cp local.properties.example local.properties  
# adicione seu tmdb.token= no local.properties  
# Sync Gradle → Run androidApp
```

Entrega: fork → PR `feat(a6-kmp): <login>` · Prazo: canvas

Atividade 4 — PWA: filmes instalável + offline (15 pts)

Stack: React + Vite + Workbox + IndexedDB (`idb`) + TMDB REST

Dois TODOs pra implementar:

```
src/services/  
  tmdb.ts    ← ✏️ TODO 1: fetchPopularMovies() (12 pts)  
  db.ts      ← IndexedDB já implementado (saveMovies/loadMovies)  
  
src/repositories/  
  moviesRepository.ts ← NetworkFirst/CacheFirst/SWR já prontos  
  
src/__tests__/unit/  
  02-tmdb-service.test.ts ← ✅ AVALIATIVO — fica vermelho até TODO 1  
  03-useFavorites.test.ts ← ✅ TODO 2: implemente os it.todo() (3 pts)
```

Entrega: fork → PR `feat(a4-pwa): <login>`

A4 — Atualize seu fork antes de começar

```
# 1. Adicionar upstream (só na primeira vez)
git remote add upstream https://github.com/jacksonsmith/puc-iec-mobile-multiplataforma.git

# 2. Buscar mudanças do prof
git fetch upstream

# 3. Mergear na sua branch main
git checkout main
git merge upstream/main

# 4. Agora crie sua branch de trabalho
git checkout -b feat/a4-pwa
```

Windows: mesmos comandos — Git Bash ou WSL.

A4 — Setup (3 passos)

```
# Passo 0 — confirmar que está na pasta certa
cd exercicios/07-pwa-tmdb/pratica
ls src/services           # deve listar: tmdb.ts db.ts
```

Windows: `cd exercicios\07-pwa-tmdb\pratica` e `dir src\services`

```
# Passo 1 — instalar dependências
npm install

# Passo 2 — configurar token TMDB
cp .env.example .env.local      # Windows cmd: copy .env.example .env.local
# edite .env.local → VITE_TMDB_TOKEN=<seu-token-aqui>
# Gere em: themoviedb.org/settings/api → API Read Access Token

# Passo 3 — rodar
npm run dev                    # dev server: http://localhost:5173
```

Vite — comandos que você vai usar

Comando	O que faz
<code>npm run dev</code>	Servidor de desenvolvimento (hot reload)
<code>npm run build</code>	Build de produção (gera <code>dist/</code>)
<code>npm run preview</code>	Serve o build de <code>dist/</code> em http://localhost:4173
<code>npm test</code>	Roda testes unitários (Vitest)
<code>npm run test:watch</code>	Vitest em modo watch (re-roda ao salvar)

Para testar o Service Worker offline você **precisa** do `build + preview` — o SW não roda em `dev` .

A4 — Rodando os testes

```
# Na pasta pratica/ (confirme com: ls src/services)
npm test
```

Saída esperada no scaffold (TODO 1 não implementado):

```
FAIL src/__tests__/unit/02-tmdb-service.test.ts
  ✗ 1. retorna lista de filmes via tmdbClient
  ✗ 2. propaga erro quando tmdbClient lança
```

Saída esperada depois do TODO 1:

```
PASS src/__tests__/unit/02-tmdb-service.test.ts
  ✓ 1. retorna lista de filmes via tmdbClient
  ✓ 2. propaga erro quando tmdbClient lança
```

Teste vermelho = implementar. Verde = entregar.

A4 — O que implementar (TODO 1)

Arquivo: `src/services/tmdb.ts`

```
// STUB atual - faz os testes falharem ↓
export async function fetchPopularMovies(_page = 1): Promise<MoviesResponse> {
  throw new Error('TODO 1: fetchPopularMovies não implementada');
}

// Sua implementação - substitua o throw ↓
export async function fetchPopularMovies(page = 1): Promise<MoviesResponse> {
  const { data } = await tmdbClient.get<MoviesResponse>('/movie/popular', {
    params: { language: 'pt-BR', page },
  });
  return data;
}
```

`tmdbClient` é um axios já configurado com `baseUrl` e `Authorization: Bearer` no arquivo.

A4 — Testando offline (demo ao vivo)

Demo rodando: <https://pwa-tmdb-aula4.vercel.app> — abra antes de implementar.

Depois do TODO 1 implementado:

1. `npm run dev` → abre <http://localhost:5173>
2. Deixe carregar os filmes (NetworkFirst busca + salva no IndexedDB)
3. Clique  **Offline** no toggle da tela → app recarrega
4. Os filmes continuam aparecendo (**cache IndexedDB**)
5. Clique  **Limpar cache** → estado vazio (sem rede + sem cache)
6. Clique  **Online** → reload → fetch fresco novamente

Por que o app não quebra offline? `moviesRepository.networkFirst()` tenta a rede, se falha vai pro IndexedDB, se também está vazio retorna `[]` (view mostra estado vazio, não erro).

PWA — por que os dados não vêm só do Service Worker?

Duas camadas de cache neste app:

Camada	O que cacheia	API
Workbox (SW)	Respostas HTTP completas (headers + body)	CacheStorage
IndexedDB (idb)	Objetos JS tratados (só results[])	indexedDB

Por que as duas?

- SW cache = resposta HTTP bruta → bom pra filmes estáticos, ruim pra paginar
- IndexedDB = dados por page-N → controle fino, offline sem SW
- Juntos: SW intercepta requests HTTP, IndexedDB guarda estado da lista

Este app usa as mesmas estratégias do slide (NetworkFirst/CacheFirst/SWR) mas na camada de dados.

PWA dentro do nativo — é possível?

```
// Flutter: WebView abre uma PWA como se fosse nativo
import 'package:webview_flutter/webview_flutter.dart';

WebViewController()
  ..setJavaScriptMode(JavascriptMode.unrestricted)
  ..loadRequest(Uri.parse('https://meupwa.app'));
```

```
// KMP/Android: mesma abordagem
val webView = WebView(context)
webView.settings.javaScriptEnabled = true
webView.loadUrl("https://meupwa.app")
```

Hybrid shell: app nativo no store + PWA dentro da WebView.

Padrão formalizado pelo **Ionic Capacitor** (capacitorjs.com) — usado em produção por milhares de apps na App Store e Google Play. `webview_flutter` é a implementação Flutter do mesmo conceito.

PWA — o que funciona (casos reais)

Todos os números abaixo são dados de engenharia publicados pelas próprias empresas — leia com olhos críticos: são cases de sucesso, não estudos controlados.

Pinterest (*web.dev case study, 2017*)

→ 60% mais engajamento · 44% mais receita de anúncios · 40% menos tempo de interação ↓

AliExpress (*web.dev case study, 2016*)

→ 104% mais conversão em novos usuários · 74% mais tempo de sessão no iOS

Twitter Lite (*Eng Blog, 2017 — arquivado*)

→ 65% mais páginas/sessão · 75% mais tweets enviados · bounce rate –20%

Padrão: são casos de *mobile web* melhorado (não nativo substituído). Ganhos são reais pra usuários que já estavam no browser — não comparam com native app direto.

PWA — o que não funciona (limitações reais)

iOS/Safari (até 2023):

- Sem push notifications nativas (adicionado só no iOS 16.4, 2023)
- Sem Background Sync
- Service Worker sem suporte à Periodic Background Sync
- PWA instalada não aparece na App Store (sem discovery)

Distribuição:

- App Store tem ~500M visitas/mês de descoberta orgânica
- PWA depende de URL direta ou search — sem equivalente

Capacidades nativas:

- Bluetooth, NFC, contacts, câmera avançada → ainda limitados ou quebrados em browsers

Fonte honesta: MDN Web Docs + Apple WebKit Bug Tracker. Limitações do iOS são documentadas pela Apple.

PWA vs Nativo — quando escolher

✓ PWA faz sentido

- Usuário base mobile-web
- Conteúdo editorial / e-commerce leve
- Deploy instantâneo sem store review
- Cross-platform com um codebase
- App secundário / fluxo isolado

✗ Nativo faz mais sentido

- Bluetooth, NFC, ARKit/ARCore
- Push em iOS confiável (pré-iOS 16.4)
- App Store discovery é parte do produto
- Gaming ou AR/VR intensivo
- Integração deep com SO (widgets, live activities)

E se quiser os dois? Flutter + PWA

A escolha **não é binária**. Padrão real usado em produção:

Flutter (ou RN) como shell → WebView carregando o PWA



- Splash e animações: **100% nativo** (Dart)
- Conteúdo e lógica: **100% web** (React + SW + IDB)
- Usuário **não sabe** que saiu do Flutter

O User-Agent `CineHub/1.0 FlutterShell` permite ao PWA detectar o contexto e adaptar comportamento.

Ionic Capacitor (capacitorjs.com) popularizou este padrão para devs web. O demo usa a mesma arquitetura via `webview_flutter`.

Demo ao vivo — Flutter Shell → PWA

```
cd exercicios/07-pwa-tmdb/demo/flutter-pwa-bridge
flutter pub get
flutter run
```

O que observar:

1. Splash Flutter nativo (animação Dart) → 2s
2. Fade para o WebView (transição suave)
3. PWA carrega com tab bar, cards, favoritos
4. ✈️ Modo avião → PWA ainda funciona (Service Worker + cache!)
5. DevTools → Application → Service Workers: registrado dentro do WebView

Ponto pedagógico: o Service Worker registra e faz cache mesmo dentro de um WebView Flutter — offline funciona de graça.

Pitfalls comuns

- ✗ `skipWaiting()` sem cuidado — clientes abertos podem ficar inconsistentes
- ✗ Cache infinito — esquecer de limpar caches antigos no `activate`
- ✗ `localStorage` ao invés de `IndexedDB` — limite de 5MB e síncrono
- ✗ Service Worker em scope errado — não intercepta o que deveria
- ✗ Não testar em "modo airplane" — só descobre bug em produção
- ✗ Assumir suporte iOS igual ao Chrome — verificar sempre em Safari/iOS

Leituras obrigatórias (pré-Aula 5)

Públicas e gratuitas:

- Archibald, J. — *The Offline Cookbook* · web.dev/offline-cookbook · Google, 2014 (atualizado)
- Workbox docs — *Caching Strategies* · developer.chrome.com/docs/workbox/caching-strategies-overview
- MDN — *Service Worker API* · developer.mozilla.org/pt-BR/docs/Web/API/Service_Worker_API

Hybrid shell (padrão do demo):

- Ionic — *Capacitor: native runtime for web apps* · capacitorjs.com (mesmo padrão do demo; versão para devs web em vez de Flutter)
- `webview_flutter` — pub.dev/packages/webview_flutter (pacote usado no demo)

Casos de uso (dados de indústria — leia criticamente):

- Pinterest PWA case study · web.archive.org — Google Showcase 2017
- AliExpress PWA case study · web.dev/aliexpress
- Twitter Lite Eng Blog 2017 · web.archive.org

Livros (biblioteca PUC / O'Reilly):

- Ater, T. — *Building Progressive Web Apps* (O'Reilly, 2017)

Próxima aula (08/07)

Aula 5 — KMP + BFF, GraphQL e Segurança

Pré-requisito:

- Atividade 4 (PWA) entregue — prazo **08/07** no Canvas
- Capstone CineHub clonado: `compartilhado/capstone/server/` → `npm run seed && npm start`

| KMP (Atividade 6) será feito **ao vivo na Aula 5** — não precisa configurar antes.

Obrigado!

 jackson.96@gmail.com

 linkedin.com/in/3jacksonsmith

 github.com/jacksonsmith

Próxima quarta, 08/07, 19h